

UNIVERSITY OF PARMA

DEPARTMENT OF MATHEMATICAL, PHYSICAL
AND COMPUTER SCIENCES

Master's Degree in Computer Science

CONSTRAINT PROGRAMMING COURSE

Project: 16 - Trucks problem

Modeling, resolution, and benchmark

Working group:

Samuel Seligardi (Student ID: 386xxx)

Luca Marchesi (Student ID: 392xxx)

Francesco Minei (Student ID: 393xxx)

Academic Year 2025/2026

Contents

1	Base problem	2
1.1	Model	2
1.1.1	State representation	2
1.1.2	Transition logic: the composite timestep	3
1.2	Implementation	3
1.2.1	Key constraints	3
1.2.2	Implementation choices	4
1.3	Results	5
1.3.1	Solver Comparison	5
1.3.2	Search Strategy Analysis	6
1.3.3	Scaling Behavior	7
2	Minimization problem	7
2.1	Model	7
2.1.1	Distance Matrix	8
2.1.2	Cost Function	8
2.2	Implementation	8
2.2.1	Reification of costs	8
2.2.2	Feasibility preservation	8
2.2.3	Weighted cross_ K decision	9
2.2.4	Distance matrix generation	9
2.2.5	Two-phase approach	9
2.2.6	Solver configuration	9
2.3	Results	9
2.3.1	Cost Quality	10
2.3.2	Runtime Breakdown	11
2.3.3	Search Strategy Evaluation	11
2.3.4	Minimization Phase Performance Summary	12
3	Conclusions	12
3.1	Key Findings	13
3.2	Model Evolution	13
3.3	Practical Implications	14
3.4	Future Directions	14

1 Base problem

The problem is defined on an undirected graph $G = (V, E)$, where h identical objects are distributed over a set of source nodes $S \subseteq V$. A truck, initially located at node $u \in V$, aims to transport all objects to a set of destination nodes $T \subseteq V$. The system is subject to capacity and movement constraints:

- The truck can carry at most two objects simultaneously.
- If the truck is fully loaded (already carrying two objects), it cannot traverse nodes that contain other objects.

The allowed actions at each step are loading an object, unloading one, or moving between adjacent nodes (*cross*). The goal of the base problem is to determine whether a valid plan of length exactly k exists that leads the system from the initial state to a final state where all target nodes T contain at least one object.

1.1 Model

The problem is modeled as a CSP (Constraint Satisfaction Problem) over a discrete time horizon. The core of the model relies on a state-space representation that evolves through discrete timesteps, where each step consists of a sequence of logistical actions.

1.1.1 State representation

The entire state of the system at any given moment is represented by a decision variable *World* matrix. This matrix has dimensions $(N + 1) \times (k + 1)$, where N is the number of nodes in the graph and k is the plan length. The structure of the matrix is defined as follows:

- **Row 0 (The truck):** the first row ($i = 0$) represents the truck itself. The value $World[0, t]$ indicates the number of objects currently carried by the truck at timestep t (at the end of the t -th transition).
- **Rows 1 to N (The nodes):** each subsequent row i corresponds to a physical node in the graph. The value $World[i, t]$ represents the number of objects located at node i at timestep t .
- **Columns (Time):** each column t represents a snapshot of the system state after the t -th transition.

This representation allows us to enforce the conservation of mass naturally: the sum of all elements in a column t must always equal the total number of objects h . While the *World* matrix tracks the quantities, a sequence of decision variables $S_{pos}[t]$ represents the truck's position. In detail:

- **Definition:** let $S_{pos}[t] \in V$ denote the node index where the truck is located at time t .
- **Initial state:** at $t = 0$, the position is fixed to the starting node u ($S_{pos}[0] = u$).
- **Trajectory consistency:** the sequence $S_{pos}[0], \dots, S_{pos}[k]$ represents a valid path on the graph, meaning that consecutive positions must be consistent with the graph's topology and the movement actions performed (as defined in the model).

1.1.2 Transition logic: the composite timestep

Instead of modeling atomic actions (like "move", "load", "unload") as separate timesteps which would unnecessarily increase the horizon $State$ we model a composite timestep. Let $State_t$ denote the system configuration at the end of timestep t , defined as the tuple $(S_{pos}[t], World[\cdot, t])$. Each transition from $t - 1$ to t consists of a strictly ordered sequence of four logical phases:

$$State_{t-1} \rightarrow Cross_{\epsilon}^* \rightarrow L^* \rightarrow Cross_{\kappa} \rightarrow U^* \rightarrow State_t$$

1. **Phase 1: Cross empty** ($Cross_{\epsilon}$) If the truck is empty at the beginning of the state ($World[0, t - 1] = 0$), it is allowed to perform an arbitrary sequence of edge crossing towards a node containing objects. Since we have a connected undirected graph this move can be seen as a teleportation. Specifically, the exact path taken is irrelevant to the model. If the truck is already carrying objects, this phase is skipped, and loading must occur at the current location.
2. **Phase 2: Load** (L^*) The truck can load one or more objects from the current node. The state variables are updated intermediately: objects are subtracted from the node and added to the truck.
3. **Phase 3: Cross loaded** ($Cross_K$) This is the mandatory movement that consumes the physical timestep. The truck, now carrying a payload, moves across an edge of the graph from the loading node to a destination node. This movement must strictly respect the graph's adjacency matrix.
4. **Phase 4: Unload** (U^*) Upon arriving at the destination, the truck may unload one or more objects. The final state at t reflects the distribution of objects after this unloading phase.

1.2 Implementation

1.2.1 Key constraints

The model is built upon a set of fundamental constraints that enforce the physical and logical rules of the problem:

Conservation constraint The total number of objects in the system remains constant throughout all timesteps. This is expressed as:

$$\sum_{i=0}^N World[i, t] = h \quad \forall t \in [0, k]$$

where h is the total number of objects. This constraint ensures that objects are neither created nor destroyed during the plan execution.

Capacity constraint The truck has a hard capacity limit of 2 objects, enforced at all timesteps:

$$World[0, t] \leq 2 \quad \forall t \in [0, k]$$

This constraint directly limits the first row of the state matrix, preventing the truck from carrying more than two objects simultaneously.

Adjacency constraint The mandatory movement phase (Cross_K) must respect the graph topology. For each timestep t , if the truck moves from node $\text{load_node}[t]$ to node $S_{pos}[t]$, then:

$$\text{Adj}[\text{load_node}[t], S_{pos}[t]] = 1$$

where Adj is the adjacency matrix of the undirected graph. This ensures that the truck only traverses edges that exist in the graph.

Cross_c teleportation When the truck is empty at the beginning of a timestep ($\text{World}[0, t-1] = 0$), it can move to any node containing objects without consuming the timestep. This is modeled as a "teleportation" action: the truck appears at a loading node $\text{load_node}[t]$ where objects are available for pickup. The specific path taken is irrelevant, as only the final position matters.

Full truck blocking rule A critical constraint enforces that a fully loaded truck cannot simply pass through occupied nodes. Formally, if the truck arrives at destination node $S_{pos}[t]$ carrying 2 objects, and $\text{World}[S_{pos}[t], t-1] > 0$ (the node was occupied before unloading), then:

$$\text{unloaded}[t] \geq 1$$

This forces the truck to unload at least one object before it can leave, preventing traversal of occupied nodes while maintaining full capacity. Furthermore, at the following timestep, the truck cannot load again until it has moved to a different node.

Goal constraint At the final timestep k , all target nodes in T must contain at least one object:

$$\text{World}[i, k] \geq 1 \quad \forall i \in T$$

This ensures that the plan successfully achieves the objective of delivering objects to all designated destinations.

1.2.2 Implementation choices

From plan_length to incremental k approach The initial model design included a decision variable plan_length representing the actual length of the plan, with the objective function:

$$\text{minimize } \text{plan_length} \text{ subject to } \text{plan_length} \leq k$$

The goal was to find the shortest valid plan within the maximum horizon k . However, the benchmark methodology employs an incremental approach: it tests $k = 1, 2, 3, \dots, k_{max}$ sequentially, stopping at the first value where a solution exists, with a timeout of 300 seconds per instance. A key insight emerged: if a solution exists for some $k = y < x$, the incremental search would already find it during iteration $k = y$. When the solver reaches $k = x > y$, any feasible plan already represents the minimal length plan for that instance. Therefore, the plan_length variable becomes redundant, minimizing it provides no additional value. The final implementation removes the plan_length variable entirely and uses a pure satisfiability approach with `solve satisfy`. This design decision:

- Reduces the search space by eliminating an unnecessary optimization variable

- Simplifies the model, making it easier for the solver to reason about
- Avoids redundant optimization when the incremental framework already identifies minimal plans
- Improves runtime by focusing purely on feasibility rather than optimality

Solver Selection Extensive testing was conducted with four different solvers: Chuffed, Gecode, HiGHs, and OR-Tools. Chuffed demonstrated significantly superior performance on this scheduling problem, consistently achieving lower runtimes and solving all 100 benchmark instances without timeout.

The rationale for Chuffed’s effectiveness lies in its *lazy clause generation* mechanism, which combines conflict-driven clause learning from SAT solving with constraint propagation. Scheduling problems like the this one involve complex temporal dependencies and resource constraints that benefit greatly from learned clauses. In contrast, traditional CP solvers like Gecode rely primarily on propagation and search heuristics, which are less effective at identifying and exploiting patterns in the constraint structure. HiGHs and OR-Tools were also evaluated to provide a comprehensive comparison. While OR-Tools achieved a 100% success rate, its average runtime was approximately twice that of Chuffed, confirming Chuffed as the optimal solver choice for this problem class.

Constraint Optimization The final model incorporates several optimizations to leverage Chuffed’s strengths:

- Heavy use of reification to model conditional constraints (e.g., *had_cross_e[t]*, *must_unload[t]*)
- Decomposition of complex transitions into atomic phases with intermediate state variables
- Explicit modeling of load/unload quantities to enable better propagation
- Channeling constraints between S_{pos} and *World* matrix to strengthen propagation

1.3 Results

This section presents the experimental evaluation of the base problem across four solvers and multiple search strategies. All benchmarks were conducted on a suite of 100 instances with varying graph sizes ($N = 10, 14, 18, 22, 26$) and plan lengths ($k = 5, 8, 11, 14, 17$). Each instance was subject to a 300-second timeout.

1.3.1 Solver Comparison

Table 1 summarizes the performance of all four solvers on the base problem.

Table 1: Solver performance comparison on base problem (100 instances, 300s timeout)

Solver	Solved	Timeouts	Avg Time (s)	Success %
Chuffed	100/100	0	1.85	100.0%
Gecode	86/100	14	53.49 (13.36)	86.0%
HiGHs	75/100	25	93.70 (24.93)	75.0%
OR-Tools	100/100	0	3.96	100.0%

Chuffed emerges as the clear winner, solving all instances with an average runtime of just 1.85 seconds. OR-Tools also achieves a perfect success rate but with double the average runtime of Chuffed (3.96s). The high timeout rates for Gecode (14%) and HiGHs (25%) indicate that these solvers struggle with the harder instances in the benchmark suite, particularly those involving larger graphs and longer plan horizons.

For Gecode and HiGHs, the values in parentheses represent the average runtime computed only over successfully solved instances, excluding timeouts. This provides a fairer comparison of solver efficiency when the success rates differ significantly. Considering only successful instances, Chuffed (1.85s) is approximately 7 times faster than Gecode (13.36s) and over 13 times faster than HiGHs (24.93s). The non parenthesized values include timeouts counted as 300 seconds, resulting in higher overall averages of 53.49s for Gecode and 93.70s for HiGHs.

Figure 1 visualizes the runtime distribution across all instances for each solver, including timeouts (shown as 300-second bars). The plot clearly shows Chuffed’s consistently low runtimes and tight distribution, while Gecode and HiGHs exhibit much higher variance, median runtimes, and multiple timeout instances.

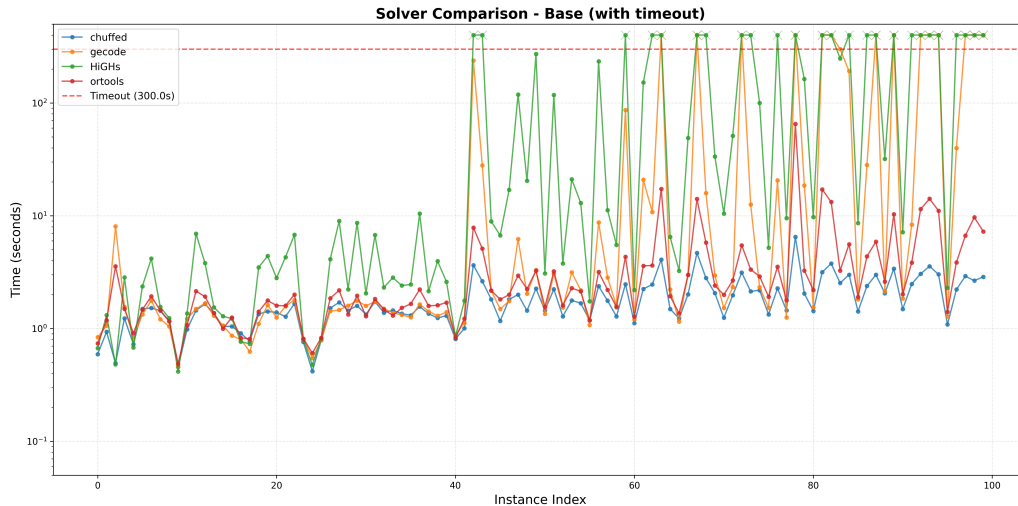


Figure 1: Solver comparison on base problem (including timeouts)

1.3.2 Search Strategy Analysis

Beyond solver selection, we evaluated the impact of custom search strategies on Chuffed’s performance. Figure 2 compares 12 different search annotation strategies against Chuffed’s default configuration.

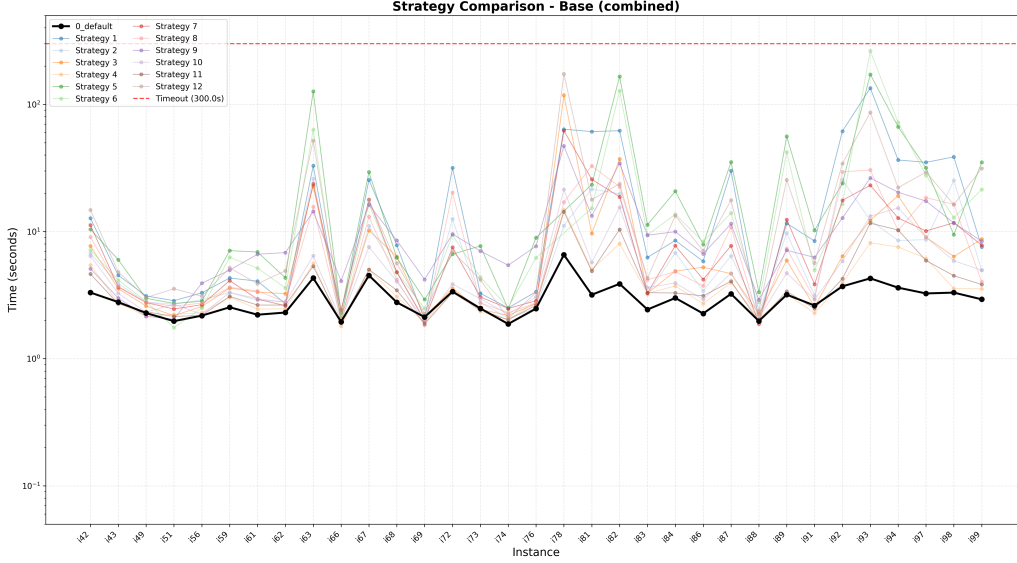


Figure 2: Search strategies comparison on base problem (including timeouts)

The results reveal a somewhat surprising conclusion: Chuffed’s default search strategy performs competitively with, and often outperforms, manually designed custom strategies. Strategies such as sequential-timestep ordering, position-first variable selection, and adaptive domain splitting show marginal differences from the default, with no strategy providing a consistent, significant improvement. This finding suggests that Chuffed’s internal heuristics and lazy clause generation mechanism are already well-tuned for this problem structure. The solver automatically learns and exploits patterns during search, reducing the need for hand-crafted annotations. For practitioners, this means the default configuration is a robust starting point, eliminating the time-consuming process of strategy tuning.

1.3.3 Scaling Behavior

An analysis of runtimes across different graph sizes reveals that performance degrades as the problem scale increases. Instances with $N=10$ nodes typically solve in under 2 seconds with Chuffed, while instances with $N=26$ nodes can take up to 10-20 seconds. The timeout instances for Gecode and HiGHs are predominantly concentrated in the larger graph sizes ($N \geq 18$), confirming that Chuffed’s lazy clause generation provides a distinct advantage on harder, more structured instances.

2 Minimization problem

The minimization problem extends the base formulation by introducing a cost function over the plan. Given a fixed plan length k (typically the minimum value found by the base problem), the objective is to find a valid plan that minimizes the total travel cost.

2.1 Model

The minimization model builds upon the base problem’s state representation and transition logic, adding a cost structure to quantify the "expense" of movements.

2.1.1 Distance Matrix

A key addition is the distance matrix $Dist[i, j]$ for all $i, j \in \{1, \dots, N\}$, representing the shortest path length between any pair of nodes. This matrix is precomputed using the Dijkstra algorithm on the graph's adjacency matrix and stored in the instance's `.dzn` file. The distance metric provides a natural cost function: moving between non-adjacent nodes incurs a cost proportional to the shortest path connecting them.

2.1.2 Cost Function

The total cost of a plan is composed of two types of movements:

Cross_e Cost When the truck is empty and "teleports" to a loading node, the cost is the shortest path distance:

$$cost_cross_e[t] = Dist[S_{pos}[t-1], load_node[t]]$$

Cross_K Cost The mandatory loaded movement incurs a weighted cost:

$$cost_cross_k[t] = Dist[load_node[t], S_{pos}[t]] \times cross_k_weight$$

where $cross_k_weight = 2$. The weight factor reflects the intuition that moving with a payload is more "expensive" than empty movement, it represents the actual physical traversal that consumes the timestep and must respect adjacency constraints.

Total Cost The objective to minimize is the sum of all movement costs across the plan:

$$total_cost = \sum_{t=1}^k (cost_cross_e[t] + cost_cross_k[t])$$

The optimization problem is formulated as:

$$\text{minimize } total_cost \text{ subject to all base problem constraints}$$

2.2 Implementation

2.2.1 Reification of costs

The cost computation relies on auxiliary reification variables to handle conditional logic. For each timestep t : $cost_cross_e[t]$ and $cost_cross_k[t]$ are computed using element constraints and multiplication. These variables enable the solver to reason about cost implications during search, allowing propagation of cost bounds even before all variables are fixed.

2.2.2 Feasibility preservation

Critically, the minimization model retains the exact same constraint set as the base problem. The cost function is purely additive, it does not alter the feasibility criteria. This ensures that any valid plan from the base problem remains valid in the minimization model, and the optimization process searches only within the space of feasible plans.

2.2.3 Weighted cross_ K decision

The choice of $cross_k_weight = 2$ reflects a design decision to prioritize minimizing loaded movements over empty movements. In practical logistics scenarios, loaded movements represent the core operational cost (fuel, time, wear) since they must follow the graph edges and consume the primary timestep resource. Empty movements, while still incurring distance costs, are modeled as "cheaper" actions.

2.2.4 Distance matrix generation

The distance matrix is generated by the `create_instances.py` script using the Dijkstra algorithm. For each instance, the script:

1. Constructs the adjacency matrix from the graph structure
2. Assigns a random weight between 1 and a fixed $maximum_weight$ to each edge
3. Applies Dijkstra to compute all-pairs shortest paths
4. Outputs $Dist[i, j]$ in the `.dzn` file alongside other instance parameters

This precomputation ensures that distance lookups during solving are instantaneous, avoiding the need for on-the-fly pathfinding within the constraint model.

2.2.5 Two-phase approach

The workflow for solving an instance combines both problems:

1. **Phase 1 (BASE):** Run the base problem with incremental k to find the minimum plan length k_{min} where a solution exists
2. **Phase 2 (MIN):** Run the minimization problem with $k = k_{min}$ to find the cost-optimal plan of that length

This two-phase strategy is efficient because the Base phase quickly identifies the shortest possible plan, and the MIN phase refines it to find the best routing within that length.

2.2.6 Solver configuration

All four solvers (Chuffed, Gecode, HiGHs, OR-Tools) were tested on the minimization problem. The solver configurations remain identical to the base problem, ensuring a fair comparison of how each solver handles the added optimization objective.

2.3 Results

The minimization problem introduces a significantly harder computational challenge: not only must the solver find a feasible plan, but it must also prove optimality of the cost function. Table 2 summarizes the performance of all four solvers on the minimization benchmark.

The SAT_MIN column represents instances where the solver found at least one feasible solution with a cost. This includes both proven optimal solutions (Optimal column) and solutions that may be suboptimal cases where the solver found a solution but could not prove optimality within the time and memory limits. Instances outside SAT_MIN (i.e.,

100 – SAT_MIN) encountered memory errors (ERROR_MIN) during the minimization phase, preventing the solver from finding any solution at all.

Table 2: Solver performance on minimization problem (100 instances, 300s timeout per phase)

Solver	Optimal	SAT_MIN	Avg Cost	MIN (s)
Chuffed	100/100	100/100	50.5	2.52
Gecode	82/100	92/100	47.9	61.54 (9.17)
HiGHs	67/100	91/100	50.3	102.83 (45.36)
OR-Tools	92/100	92/100	47.5	39.63 (16.61)

Chuffed again demonstrates exceptional performance, achieving 100% proven optimality (100 SAT_MIN, 100 optimal) with an average minimization phase runtime of just 2.52 seconds and no memory errors. This is significantly faster than all other solvers: approximately 3.6 times faster than Gecode’s proven optimal instances average (9.17s), 18 times faster than HiGHs’ proven optimal instances (45.36s), and 6.6 times faster than OR-Tools’ proven optimal instances (16.61s). The optimization phase proves that Chuffed’s lazy clause generation excels not only at finding feasible solutions but also at proving cost optimality efficiently.

Gecode found solutions for 92 instances but proved optimality for only 82, encountering 8 memory errors and 10 instances with potentially suboptimal solutions (where a solution was found but optimality could not be proven within the timeout). Its average MIN phase time is 61.54 seconds including failed instances (9.17 seconds for proven optimal instances), still nearly 4 times slower than Chuffed even on successful cases. HiGHs struggles significantly on this optimization problem, finding solutions for 91 instances but proving optimality for only 67, with 9 memory errors and 24 potentially suboptimal solutions. Its average MIN time is 102.83 seconds including failed instances (45.36 seconds for proven optimal instances). OR-Tools achieves a 92% proven optimality rate (92 SAT_MIN, all proven optimal, with 8 memory errors on remaining instances) with an average MIN time of 39.63 seconds including failed instances (16.61 seconds for proven optimal instances), but remains substantially slower than Chuffed.

2.3.1 Cost Quality

An interesting observation is the variation in average costs across solvers. The solutions found by Chuffed have an average cost of 50.5, while Gecode’s average is 47.9, HiGHs’ is 50.3, and OR-Tools’ is 47.5. This discrepancy arises because the averages are computed over the SAT_MIN instances for each solver, different subsets of the 100 instances where a solution was found. Importantly, for Gecode and HiGHs, these averages include potentially suboptimal solutions, which may artificially lower or raise the average depending on the quality of non-proven solutions.

Gecode (92 SAT_MIN instances, 82 proven optimal) and OR-Tools (92 SAT_MIN instances, 92 proven optimal) both show lower average costs than Chuffed (47.9 and 47.5 vs 50.5). For OR-Tools, since all SAT_MIN solutions are proven optimal, the comparison is direct: it solved a subset of instances (missing 8 due to memory errors) that happen to have inherently lower optimal costs. For Gecode, the average includes 10 potentially suboptimal solutions, which could bias the average downward if these non-proven solutions are actually suboptimal. HiGHs (91 SAT_MIN instances, 67 optimal) has an average

cost of 50.3, nearly identical to Chuffed’s despite having 24 potentially suboptimal solutions. This suggests that either these non-proven solutions are close to optimal, or the distribution of instance difficulties balances out the effect.

2.3.2 Runtime Breakdown

Figure 3 visualizes the runtime distribution for the minimization phase across all four solvers, including timeout instances (shown as 300-second bars).

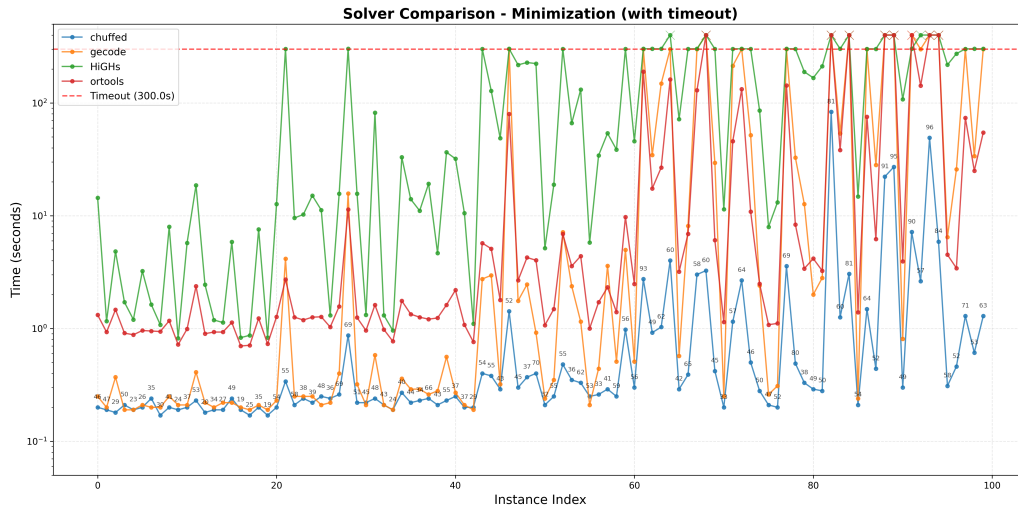


Figure 3: Solver comparison on minimization problem (including timeouts)

The plot highlights Chuffed’s tight runtime distribution, with all instances solving in under 5 seconds and achieving proven optimality. Gecode encountered 8 memory errors and found 10 potentially suboptimal solutions where optimality could not be proven within the timeout (82% proven optimal), OR-Tools had 8 memory errors with all SAT_MIN instances achieving proven optimality (92% proven optimal), and HiGHS shows 9 memory errors plus 24 potentially suboptimal solutions where optimality could not be proven (67% proven optimal).

2.3.3 Search Strategy Evaluation

Similar to the base problem, we evaluated custom search strategies against Chuffed’s default configuration. Figure 4 presents the results across all instances, including timeouts.

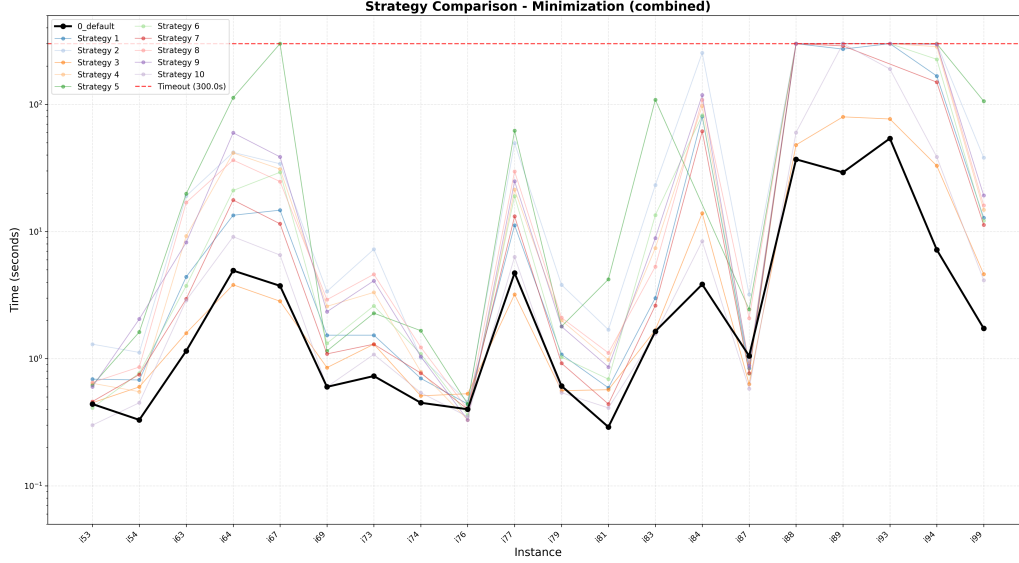


Figure 4: Search strategies on minimization problem (including timeouts)

Once again, the default search strategy proves highly effective, with no custom strategy providing a clear, consistent advantage. The optimization objective does not appear to benefit significantly from manual search annotations, further proving the effectiveness of Chuffed’s internal heuristics.

2.3.4 Minimization Phase Performance Summary

The minimization phase results clearly demonstrate Chuffed’s superiority in proving cost optimality. With an average MIN time of 2.52 seconds and a 100% proven optimality rate, Chuffed significantly outperforms all other solvers. When including failed instances, timeouts and memory errors counted as 300 seconds, Gecode averages 61.54 seconds (82% proven optimal), HiGHs averages 102.83 seconds (67% proven optimal), and OR-Tools averages 39.63 seconds (92% proven optimal).

Even when comparing only instances where optimality was proven, which provides a more favorable view of the competing solvers, Chuffed remains the clear winner: Gecode’s 9.17 seconds is $3.6\times$ slower, HiGHs’ 45.36 seconds is $18\times$ slower, and OR-Tools’ 16.61 seconds is $6.6\times$ slower than Chuffed’s 2.52 seconds. This performance advantage, combined with the perfect proven optimality rate and no memory errors, confirms that Chuffed’s lazy clause generation mechanism is exceptionally well-suited for optimization problems with complex temporal and resource constraints.

3 Conclusions

This project presented a comprehensive constraint programming approach to the Trucks problem, including both satisfiability and optimization variants. The work demonstrates the effectiveness of model design, solver selection, and evaluation while facing logistical planning problems.

3.1 Key Findings

Best Configuration Chuffed with default search settings emerged as the optimal solver configuration for both the base problem and the cost minimization variant. Its lazy clause generation mechanism proved highly effective on the temporal and resource constraints inherent in the Trucks problem, achieving 100% success rates with consistently low run-times.

Incremental K Approach The removal of the `plan_length` optimization variable in favor of a pure satisfiability model, combined with an incremental search over k values, proved to be both conceptually simpler and computationally more efficient than embedded optimization. This design decision reduced the search space and eliminated redundant work, as the incremental framework inherently finds minimal-length plans.

Solver Performance Comparison The experimental results clearly delineate solver capabilities:

- **Chuffed:** 100% success on BASE (1.85s avg), 100% optimality on MIN (2.52s avg)
- **OR-Tools:** 100% success on BASE (3.96s avg), 92% optimality on MIN (16.61s avg)
- **Gecode:** 86% success on BASE (13.36s avg), 82% optimality on MIN (9.17s avg)
- **HiGHs:** 75% success on BASE (24.93s avg), 67% optimality on MIN (45.36s avg)

Chuffed’s superiority is particularly noticeable on harder instances and optimization problems, where lazy clause learning provides a clear advantage over traditional constraint propagation.

Custom Search Strategies Contrary to initial expectations, custom search strategies, including sequential timestep ordering, position, first heuristics and adaptive domain splitting, did not provide a significant improvements over Chuffed’s default configuration. This finding suggests that Chuffed’s internal heuristics and learned clauses effectively capture the problem structure, making manual strategy tuning unnecessary for this problem class.

Two-Phase Optimization The workflow of first finding the minimum plan length via BASE, then optimizing cost at that fixed length, proved efficient and practical. The Base phase quickly identifies the shortest feasible plan, and the MIN phase refines routing decisions without re-exploring infeasible regions of the search space.

3.2 Model Evolution

The initial model included redundant optimization variables and overly complex action encoding. Through analysis of solver behavior and benchmark results, the model evolved to:

- Eliminate the `plan_length` variable in favor of incremental external search
- Strengthen propagation through reification and channeling constraints
- Leverage precomputed distance matrices for efficient cost computation

3.3 Practical Implications

For anyone facing similar planning problems, this work offers several insights:

1. **Solver choice matters:** Lazy clause generation solvers (e.g., Chuffed) should be the first choice for scheduling and planning problems with complex temporal dependencies.
2. **Default settings are robust:** Invest effort in model design rather than search strategy tuning; modern solvers often perform well out-of-the-box.
3. **Incremental approaches work:** For length minimization, external incremental search can be simpler and faster than embedded optimization.
4. **Two-phase optimization:** Separating feasibility and cost optimization can improve efficiency when plan length is a critical metric.

3.4 Future Directions

Potential extensions of this work include:

- Scaling to larger graphs ($N > 30$) and longer horizons ($k > 20$) to stress-test solver limits
- Testing with alternative cost functions (e.g. energy consumption)
- Extending the model to handle heterogeneous objects or multi-capacity trucks

In conclusion, this project demonstrates that well designed constraint models paired with appropriate solvers can efficiently solve complex planning problems. The Chuffed solver with default settings represents the best configuration for the Trucks problem, achieving perfect success rates with minimal runtimes across all benchmark instances.